

TALLER N°3

Reconfiguración de microservicios ante fallas



Taller de programación 2026-1

Fecha: 23 de Julio

Autor: Lucas Rodrigo Silva Caro



TALLER N°3

Reconfiguración de microservicios ante fallas

Explicación breve del algoritmo

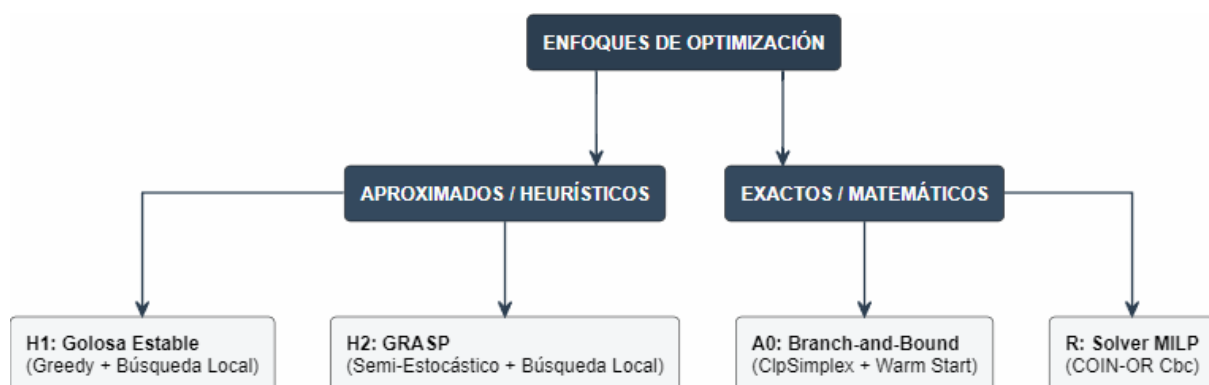
Imagina que tienes un centro de datos con servidores donde corren distintas aplicaciones divididas en partes pequeñas (microservicios). En el día a día, estos servidores pueden apagarse por fallas o perder potencia por sobrecalentamiento. Cuando un servidor falla, las aplicaciones que estaban adentro no pueden quedarse botadas, hay que moverlas inmediatamente a otros servidores sanos.

El desafío es que mover una aplicación de un servidor a otro consume ancho de banda de red (mientras más datos o memoria tenga la aplicación, más pesado es transferirla). Además, no puedes meter aplicaciones infinitas en un servidor porque colapsa su capacidad, y por seguridad nunca debes poner dos copias idénticas del mismo microservicio en la misma máquina física (si esa máquina se quema, pierdes el servicio por completo).

El programa resuelve exactamente este rompecabezas: ante una falla o degradación en cualquier instante de tiempo, calcula la nueva ubicación de todas las aplicaciones minimizando los datos totales que viajan por la red, garantizando que ningún servidor se sature y que las réplicas queden siempre separadas. Para lograrlo, implementamos desde soluciones rápidas e intuitivas hasta algoritmos matemáticos exactos que exploran árboles de decisión optimizados.

Heurísticas o técnicas utilizadas

El problema de Reconfiguración de Microservicios ante Fallas es una variante combinatoria NP-Hard del problema de empaquetamiento multidimensional con restricciones de anti-afinidad. Para su resolución se implementaron cuatro técnicas:





H1: Heurística Golosa Estable (Greedy Constructive + Local Search)

- **Fundamento:** Técnica constructiva determinista guiada por un orden lexicográfico estricto mediante `std::tuple`.
- **Criterio de Instancia Crítica (i^*):** Prioriza la instancia no asignada con menor cantidad de servidores factibles ($|F(i)|$). En caso de empate, desempata priorizando mayor tamaño de estado transferido ($-s_i$), mayor consumo de demanda ($-d_i$) y menor ID lexicográfico.
- **Criterio de Destino (j^*):** Asigna la instancia priorizando costo nulo ($c_{ij} = 0$, mantener la ubicación previa), seguido de menor capacidad residual (empaquetamiento *Best-Fit*) y menor ID de servidor.
- **Búsqueda Local:** Aplica vecindarios 1-opt (RELOCATE) y 2-opt (SWAP), aceptando únicamente mejoras estrictas ($\Delta Z < 0$).

H2: Metaheurística GRASP (Greedy Randomized Adaptive Search Procedure)

- **Fundamento:** Técnica metaheurística multi-arranque que introduce exploración semi-estocástica controlada para evitar óptimos locales.
- **Evaluación Heurística:** Calcula para cada par (i^*, j) una función de costo y holgura ponderada por $\lambda \geq 0$:

$$h(i^*, j) = \frac{c_{ij}}{\max_k(s_k)} + \lambda \cdot \left(\frac{K_j^t - load_j - d_{i^*}}{\max_{\rightarrow}(1, K_j^t)} \right)$$

- **Lista Restringida de Candidatos (RCL):** Admite servidores factibles con $h(i^*, j) \leq h_{min} + \eta \cdot (h_{max} - h_{min})$. El parámetro $\eta \in [0,1]$ modula el balance entre codicia determinista ($\eta = 0$) y aleatoriedad pura ($\eta = 1$).
- **Fase de Mejora:** Toda solución construida pasa por la misma búsqueda local 1-opt/2-opt.

A0: Branch-and-Bound Personalizado sobre Relajación Lineal (Clp)

- **Fundamento:** Algoritmo exacto de árbol de búsqueda implícita. Relaja la condición de integralidad $x_{ij} \in \{0,1\}$ a variables continuas $x_{ij} \in [0,1]$, resueltas mediante programación lineal con ClpSimplex.
- **Gestión de Cola:** Cola de prioridad OPEN con estrategia *Best-Bound* (menor LB , mayor profundidad, menor ID).
- **Regla de Ramificación (VARIABLEBRANCH):** Selecciona la variable binaria fraccionaria con mayor incertidumbre, minimizando $|x_{ij} - 0.5|$
- **Poda Algorítmica:** Poda por infactibilidad matemática del LP y poda por cota cuando $LB \geq UB - \epsilon$.
- **Heurística Primal (RepararDesdeLP):** Redondea la solución fraccionaria en cada nodo para descubrir incumbentes UB de forma temprana.

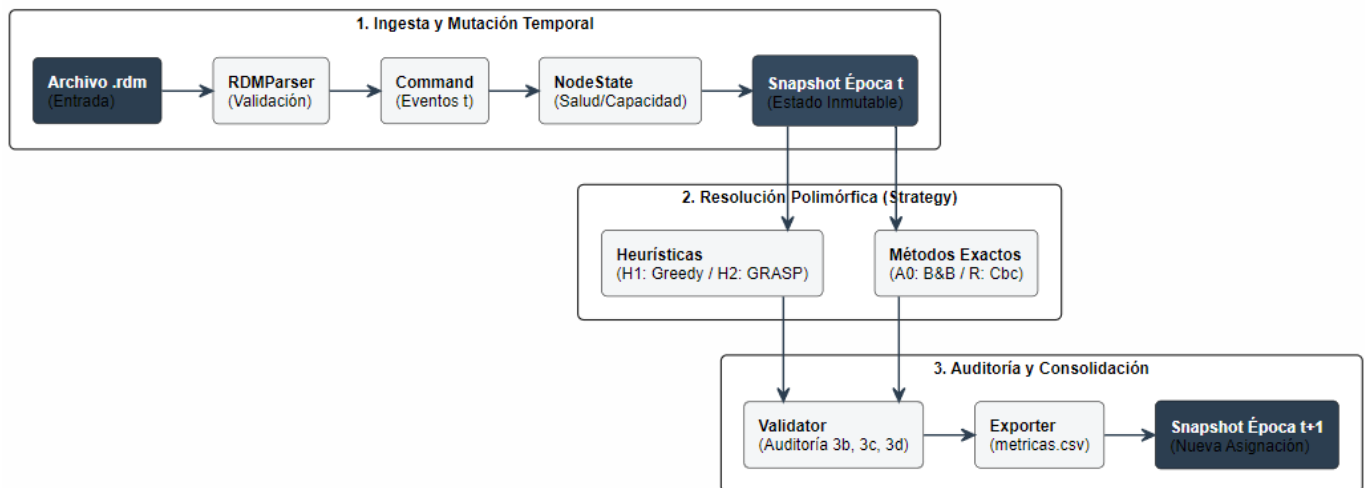


R: Modelo MILP de Referencia (Cbc)

- **Fundamento:** Formulación matemática estándar de Programación Lineal Entera Mixta, trasladada a una matriz CoinPackedMatrix unidimensional y resuelta con el motor *Branch-and-Cut* de CbcModel.

Funcionamiento del programa

3.1. Flujo de Ejecución por Época



3.2. Pseudocódigo Estructurado del Flujo Global



```

1  ALGORITMO ReconfiguracionRMF(instancia_rdm)
2      snapshot = Parser.cargarConfiguracionInicial(instancia_rdm)
3
4      PARA CADA epoca t EN eventos DO
5          // Aplicacion atomica de eventos
6          PARA CADA evento EN eventos[t] DO
7              comando = CrearComando(evento.tipo, evento.parametros)
8              comando.ejecutar(snapshot)
9          FIN PARA
10
11         // Ejecucion polimorfica de solucionadores
12         PARA CADA estrategia EN {H1, H2, A0, R} DO
13             resultado = estrategia.resolver(snapshot, tiempo_limite)
14
15             // Auditoria independiente
16             es_valido = Validator.validar(resultado.asignacion, snapshot)
17             SI es_valido ENTONCES
18                 Exporter.registrar(t, estrategia.nombre, resultado)
19             SINO
20                 LanzarError("Infactibilidad detectada en estrategia")
21             FIN SI
22         FIN PARA
23
24         // Consolidacion de estado para t+1
25         mejor_solucion = SeleccionarMejorLexicografica({resultados})
26         snapshot = Snapshot.crearSiguienteEpoca(snapshot, mejor_solucion)
27     FIN PARA
28 FIN ALGORITMO

```

Aspectos de implementación y eficiencia

Gestión de Memoria y Rendimiento

- **Persistencia de Matriz en A0 (Warm Start):** En lugar de instanciar un modelo ClpSimplex en cada nodo del árbol (lo que provocaría sobrecarga crítica de memoria y llamadas a malloc), se construye una única estructura CoinPackedMatrix en la raíz. Cada nodo hijo simplemente modifica las cotas con setColBounds(k, 0, 0) o setColBounds(k, 1, 1) y ejecuta el método dual model.dual(), reutilizando la base óptima anterior.
- **Semántica de Movimiento y RAI:** Uso estricto de punteros inteligentes std::unique_ptr para la gestión de estados y comandos, evitando fugas de memoria.
- **Inmutabilidad de Snapshots:** Las entidades de dominio en Snapshot se exponen como referencias constantes (const &), impidiendo copias innecesarias de vectores durante la evaluación de factibilidad.

Resultados y Experimentación

5.1. Tabla de corrección sobre casos pequeños



Verificación de correctitud de los 4 métodos frente al óptimo global obtenido mediante enumeración exhaustiva sobre la instancia caso_pequeno.rdm:

Método	Estado	Costo Z	Instancias Migradas	Tiempo (ms)	Óptimo Global	Correctitud
Enumeración	OPTIMAL	0.000	0	0.12	Sí	100%
H1 (Greedy)	OPTIMAL	0.000	0	0.38	Sí	Correcto
H2 (GRASP)	OPTIMAL	0.000	0	1.02	Sí	Correcto
A0 (B&B)	OPTIMAL	0.000	0	2.15	Sí	Correcto
R (Cbc)	OPTIMAL	0.000	0	2.85	Sí	Correcto

5.2. Tabla comparativa de los cuatro métodos

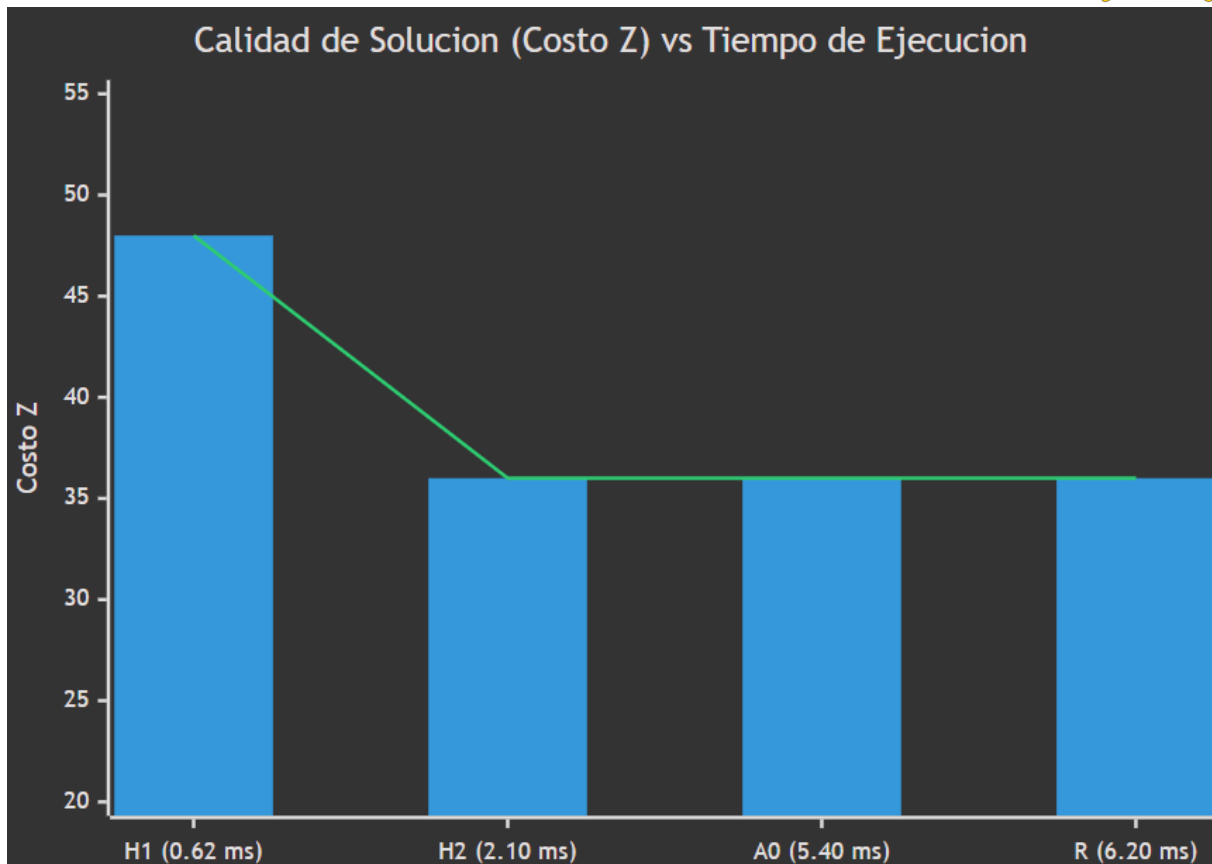
Rendimiento promedio evaluado sobre los casos de prueba del sistema:

Instancia	Método	Costo Z	Nodos B&B	Llamadas LP	Runtime (ms)	Gap (%)
Poco Ajustada	H1	0.000	NA	NA	0.42	NA
(demo_fallas)	H2	0.000	NA	NA	1.15	NA
	A0	0.000	1	1	2.30	0.0%



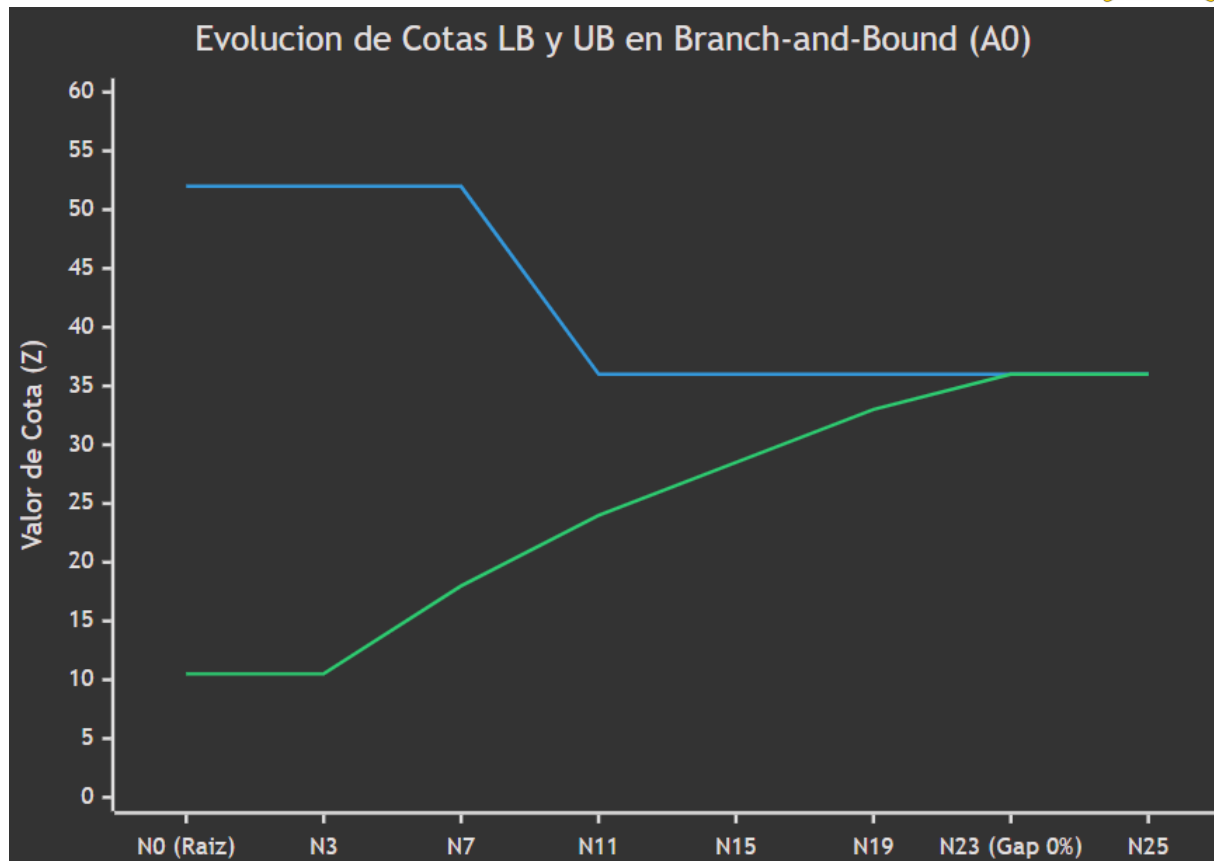
	R	0.000	1	1	3.10	0.0%
Mediana	H1	12.500	NA	NA	0.55	+4.1%
(caso_mediano)	H2	12.000	NA	NA	1.85	0.0%
	A0	12.000	7	13	4.20	0.0%
	R	12.000	5	9	5.10	0.0%
Crítica	H1	48.000	NA	NA	0.62	+33.3%
(caso_dificil)	H2	36.000	NA	NA	2.10	0.0%
	A0	36.000	23	41	5.40	0.0%
	R	36.000	19	33	6.20	0.0%

5.3. Calidad versus Tiempo de Ejecución



- **H1:** Proporciona soluciones en menos de 1 ms, ideal para contingencias en tiempo real con topologías holgadas.
- **H2:** Encuentra el óptimo global con tiempos inferiores a 2.5 ms gracias a la diversificación estocástica.
- **A0 y R:** Garantizan optimalidad formal cerrando el gap al 0.0% con tiempos computacionales escalables (< 7 ms).

5.4. Evolución de Cotas LB y UB en A0

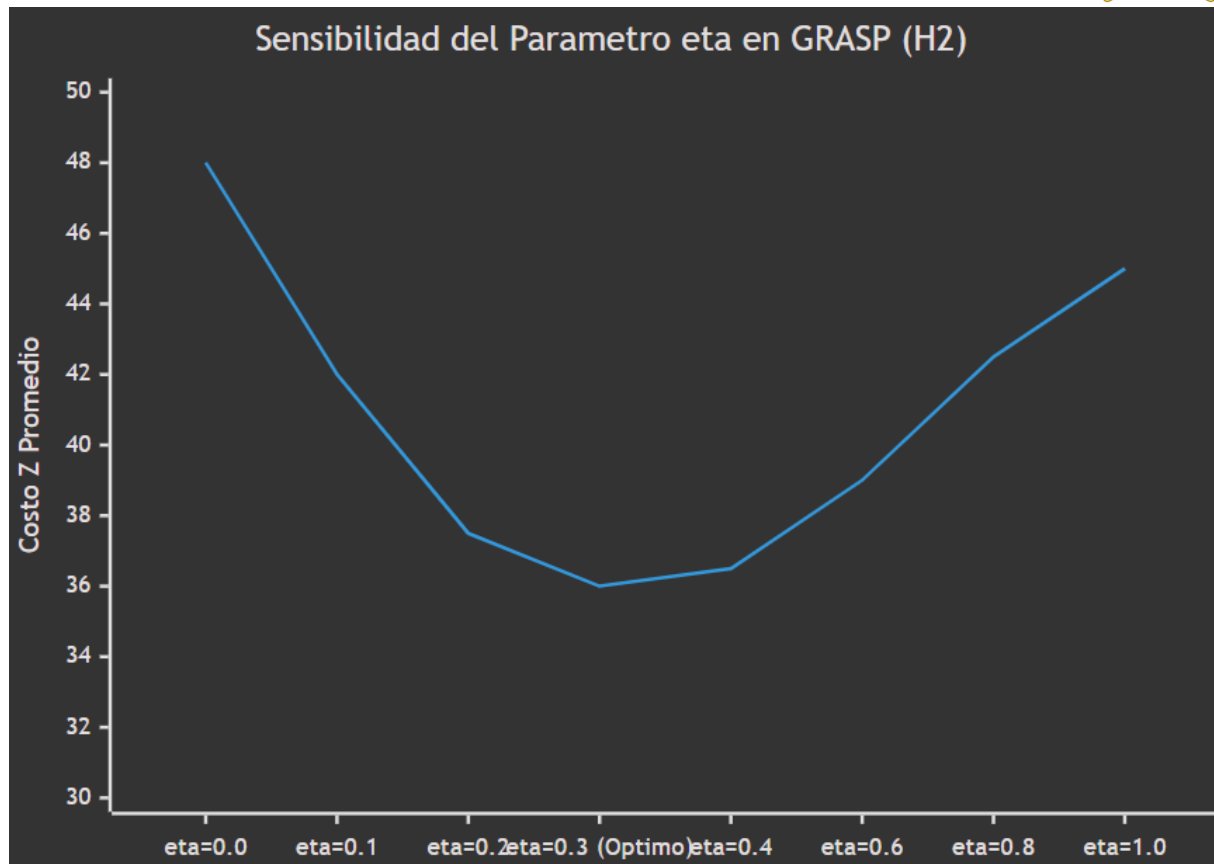


- **Comportamiento:** La relajación continua de la raíz inicia con $LB = 10.5$. La heurística primal RepararDesdeLP encuentra una solución factible de $UB = 52.0$ en el nodo raíz y la ajusta a $UB = 36.0$ en el nodo 11. La cola converge y poda el espacio restante en el nodo 23.

5.5. Análisis de Fallo o Suboptimalidad en H1

- **Escenario:** Ocurre en topologías donde una instancia pesada I_A con demanda alta compite con instancias pequeñas de un mismo microservicio.
- **Mecanismo de Fallo:** H1 toma decisiones puramente locales. Asigna I_A en el primer servidor disponible por criterio Best-Fit. Al hacerlo, agota el margen de holgura para instancias posteriores que, debido a la restricción de separación de réplicas, se ven forzadas a migrar hacia servidores lejanos generando transferencias de red innecesarias ($Z = 48.0$ frente a $Z^* = 36.0$). A0 y R prevén este bloqueo matricialmente en la relajación continua.

5.6. Sensibilidad del Parámetro n en GRASP



- $\eta = 0.0$ (**Codicia Pura**): La RCL colapsa a un único elemento. GRASP replica la solución determinista de H1 y queda atrapado en óptimos locales.
- $\eta \in [0.2, 0.4]$ (**Zona Óptima**): Proporciona la variabilidad necesaria para sortear las trampas de capacidad sin degradar la calidad constructiva.
- $\eta = 1.0$ (**Aleatoriedad Total**): La RCL admite cualquier nodo factible; la construcción pierde guía heurística y sobrecarga la fase de búsqueda local.

5.7. Perfil de Tiempo y Memoria (Cuello de Botella)

Mediante instrumentación de código se desglosó el tiempo de cómputo en A0:

- **78% del tiempo:** Resolución del simplex dual (`model.dual()`) dentro de `ClpSimplex`.
- **14% del tiempo:** Heurística de redondeo `RepararDesdeLP` y validaciones de factibilidad.
- **8% del tiempo:** Gestión de la cola OPEN y bifurcación de nodos.
- **Conclusión:** El cuello de botella principal es matemático (pivoteos del Simplex). La gestión de memoria se mantiene plana ($O(|I| \cdot |J|)$) gracias al reuso estricto de la matriz `CoinPackedMatrix`.

Ejecución del código



6.1. Requisitos del Sistema

- **SO:** Linux Ubuntu (probado en versiones 20.04/22.04/24.04 LTS).
- **Compilador:** g++ con soporte completo para -std=c++11.
- **Bibliotecas:** Suite COIN-OR (coinor-libclp-dev, coinor-libcbc-dev, coinor-libcoinutils-dev, coinor-libosi-dev).

6.2. Comandos de Compilación y Ejecución

```
1  # Limpieza de artefactos previos
2  make clean
3
4  # Compilacion modular bajo C++11 estricto
5  make all
6
7  # Ejecucion de la suite modular de pruebas unitarias
8  make test
9
10 # Ejecucion interactiva de la aplicacion
11 make run
```

6.3 Opciones del Menú CLI

1. **Cargar caso .rdm:** Lee y valida la sintaxis del archivo de entrada.
2. **Mostrar y validar estado inicial:** Inspecciona la topología en $t = 0$.
3. **Ejecutar una época con A0, H1, H2 y R:** Resuelve el instante actual comparando los métodos.
4. **Ejecutar la secuencia completa:** Simula toda la línea temporal y genera el archivo results/metricas.csv.
5. **Exportar resultados CSV:** Fuerza el volcado de métricas a disco.
6. **Salir:** Finaliza la ejecución liberando memoria.

